

Sesión 2 de Ako, contada fácil

Resumen para leer tranquilo. Todo lo del 30 de septiembre, explicado como si montáramos un bar. Hoy construimos **el archivador del bar**: donde la app guarda la carta, las mesas y los pedidos.

1. Repaso de lo que no quedó claro en la S1

- **Gradle es el jefe de cocina.** Tú escribes recetas (el código). Gradle va a por los ingredientes que faltan (las librerías, como Room), lo cocina todo en orden (compilar) y lo emplata en algo que se puede servir: el **APK**, el archivo que se instala en el móvil. Cuando pulsas Run ►, el que trabaja es Gradle.
- **El catálogo de versiones (`libs.versions.toml`) es la lista de proveedores.** Dice de qué marca y de qué lote es cada ingrediente: «Room, siempre del lote 2.8.5». Si mañana cambias de lote, lo cambias en un solo sitio y toda la cocina se entera. Ojo: no son las versiones de tu app, son las de lo que **usas**.
- **`minSdk 26` es la edad mínima para entrar al bar.** La app solo se instala en Android 8 o más nuevo. ¿Por qué? Porque el PIN se guarda **triturado** con una picadora llamada **PBKDF2 con SHA-256**, y esa picadora viene de serie en Android desde la versión 26. En un móvil más viejo, la app se cerraría al crear el PIN.

2. El archivador: seis ideas básicas

Imagina un archivador de fichas de cartulina.

- **Tabla** = un cajón (el cajón de los platos, el de las mesas...).
- **Fila** = una ficha concreta dentro del cajón: «Agua, 1,50 €».
- **Clave primaria (`id`)** = el **número de carné** de la ficha. No hay dos iguales. Aunque haya dos platos llamados «Tortilla», sus carnés son distintos.
- **Clave foránea** = una ficha que **apunta** a otra por su carné. La ficha del Agua no escribe «Bebidas»: pone «categoría nº 2».
- **RESTRICT** = «no te dejo tirar esa ficha mientras otra apunte a ella». Mientras el Agua apunte a Bebidas, Bebidas no se puede borrar.
- **Índice único** = un **portero** que no deja entrar dos fichas con el mismo valor. No puede haber dos platos con el número 7.

¿Y **CASCADE**? Es lo contrario de **RESTRICT**: «si tiras esta ficha, tira también las que apuntan a ella». En el diseño solo había una, la de los extras («+ queso») de una línea de la comanda. Como los extras son del nivel 2, **hoy en Ako no hay ninguna CASCADE: todo es RESTRICT**. En Ako nada se borra, se **elimina** marcando `activo = false`.

Room es el archivero. Tú le das el molde de la ficha y una etiqueta encima (`@Entity`, «esto es un cajón»), y él monta el cajón de verdad en el móvil. Tú no escribes SQL a mano.

3. Los paquetes: los cajones de la cocina

Creamos las carpetas del código. Un **paquete** es una carpeta con nombre, como los cajones de una cocina: cubiertos en uno, sartenes en otro.

La especial es `dominio`: es la **libreta de reglas** del bar («el total es la suma de las líneas», «el cambio es lo entregado menos el total»). Es código normal de Kotlin, pero **sin nada del móvil dentro**. ¿Para qué? Para comprobar que la libreta suma bien sin tener que encender los fogones: sus pruebas se ejecutan en el ordenador en dos segundos, sin emulador.

Truco que aprendiste a golpes: el clic derecho para crear un paquete va **sobre** `yunkang.ako`, no sobre `java`. Y al escribir el nombre no se borra el `yunkang.ako` que ya viene puesto.

4. El enum: los sellos de goma

Una comanda está en uno de tres estados: **PENDIENTE** (la mesa está comiendo), **PAGADA** (cobrada) o **ANULADA** (cancelada). Eso es un **enum**: una lista cerrada de valores, como tres sellos de goma en la caja del bar.

¿Por qué no un «sí/no»? Porque hay tres estados. ¿Por qué no un texto libre? Porque alguien escribiría «Pagada», otro «pagado» y otro «PAGDA». Con el enum, **si escribes mal PAGDA, Android Studio lo pone en rojo antes de ejecutar nada**. Con texto libre no avisa, y esa comanda desaparecería de la caja del día sin que nadie se enterara. Tus palabras: «si tenemos una errata, lo señala».

El camarero **nunca escribe un estado**: lo pone la app al pulsar Enviar, Cobrar o Anular.

Tu decisión (P113): la base de datos solo guarda números y texto, así que alguien tiene que convertir el sello en palabra. Podíamos escribir nosotros el traductor (`Converters`) o dejar que

lo haga Room, que lo trae de serie desde la versión 2.3. Elegiste que lo haga Room: «es como ir a la ciudad en bici o en coche costando lo mismo». En la base de datos se lee igual: `PAGADA`.

5. Tus reglas nuevas: el prototipo primero

A mitad de sesión decidiste algo importante:

- **Solo se construye el nivel 1.** Orden: **prototipo** → **doc** → **y solo si sobra tiempo, extras del nivel 2.**
- **Nada de código «por si acaso».** Cada línea tiene que servir para algo hoy. Por eso quitamos las 7 tablas del nivel 2 (modificadores, idiomas, etiquetas...): **de 14 tablas pasamos a 7** (P115).
- **Lo que se empieza se termina.** Tus palabras: «si pongo 1, tengo que poner los demás números».
- **El diseño del nivel 2 no se borra** de los documentos: se guarda para después.
- **Lo que decidimos aquí manda** sobre el Project de la app de Claude. Los cambios están en la lista «Para el Project» (`docs/decisiones-code.md`, apartado 6), para que el chat del Project corrija el doc.

6. Las 7 tablas

- `categoria` y `producto`: la carta. El precio se guarda en **céntimos enteros** (1,50 € = 150), porque los ordenadores fallan en la última cifra con los decimales (0,1 + 0,2 da 0,300000000000000004). Con céntimos, 150 + 250 es siempre 400.
- El `?` detrás de un tipo significa «esta casilla puede quedar vacía»: un plato puede no tener descripción, pero nunca puede no tener nombre.
- `alergeno` y `producto_alergeno`: la **tabla puente**. La Hamburguesa tiene gluten y lácteos, y el gluten está en varios platos. Como no cabe en una casilla, se hace un cajón de fichitas de dos datos: «plato 5 + alérgeno 1». Su carné es **la pareja**, así que la misma pareja no puede estar dos veces.
- `mesa`: no guarda si está ocupada. Una pizarra que dice «ocupada» miente el día que se te olvida borrarla. La mesa está ocupada **si tiene una comanda PENDIENTE**, y eso se mira cada vez. Igual con el total de la comanda: se suma de sus líneas, no se guarda.
- `comanda` y `linea_comanda`: la línea **congela** el nombre y el precio del plato al pedirlo, como un ticket ya impreso. Si mañana el Agua sube a 1,80 €, el recibo de ayer sigue diciendo 1,50 €. La casilla que lo consigue es `precio_unitario_centimos`.

- Las **fechas** se guardan como un número muy grande: los milisegundos que han pasado desde el 1 de enero de 1970. Es la forma estándar en que los ordenadores guardan un momento.

7. **AppDatabase** : el archivador entero

Es la lista de los 7 cajones y su número de versión. Tú escribes solo el esquema (`abstract` = «esta clase está incompleta») y **Room escribe el resto al compilar**. Al compilarla, Room revisó por primera vez tus 7 tablas y dio el visto bueno. Si se te olvidara una tabla en la lista, Room daría error al compilar, porque otra tabla apuntaría a un cajón que no existe.

8. **EntradaAko** y **lazy** (lo que más te llamó la atención)

Cuando tocas el icono de la app, **Android crea primero un objeto que representa la app entera** mientras está abierta (se llama `Application`), y después la pantalla. Las pantallas van y vienen (se destruyen hasta al girar el móvil), pero ese objeto se queda todo el rato. Es **el local del bar**; las pantallas son los camareros de turno.

Nosotros hicimos nuestra versión y la llamaste `EntradaAko`. Su trabajo es **guardar la llave del almacén**: la única base de datos.

¿Por qué una sola? Abrir la base de datos es caro, como encender el horno. Y si hubiera dos, sería como dos camareros con **dos libretas** de la misma mesa: al final no sabes cuál es la buena.

lazy es el gancho de la llave con una regla: «si no hay llave, se hace una; si ya hay, se da esa». Se crea **la primera vez que alguien la pide**, y a partir de ahí siempre es la misma. Y Kotlin garantiza que es una sola aunque la pidan dos tareas a la vez. Una línea hace lo que en los tutoriales ocupa doce.

Lo resumiste así: «Ako guarda el acceso a la base de datos; en la otra opción, Ako le dice a AppDatabase que mire en su caja fuerte». Elegiste la opción sencilla (P116).

Para que Android use `EntradaAko` hay que decírselo en el **manifiesto**, la ficha de la app que Android lee antes de abrirla. Si se te olvida esa línea, la app **compila igual**, pero se cierra de golpe la primera vez que alguien pide la base de datos. Es de los pocos errores que el compilador no avisa.

9. Los textos: `strings.xml`

Los nombres de los alérgenos, «Otros», «Bebidas» y «Agua» no se escriben dentro del código: van en `strings.xml`, el cajón de los textos. Así están todos juntos y, si algún día hay otro idioma, se traduce ese archivo sin tocar el código.

10. Los DAOs: el mostrador del archivero

Un **DAO** es la lista de cosas que le puedes pedir al archivero: «guarda esta categoría», «guarda estas 60 mesas». Es una **interfaz**: dice **qué** se puede pedir, no **cómo** (como la carta del bar, que dice los platos y no la receta). La receta la escribe Room.

- `@Insert` = «esto es guardar una ficha nueva».
- `suspend` = «esto puede tardar; no se puede llamar desde la pantalla», porque la pantalla se quedaría congelada esperando.
- `insertar` **devuelve el carné** que Room le ha dado a la ficha. Es el **ticket del guardarropa**: al guardar Bebidas te dan el ticket nº 2, y el Agua lo necesita para decir «soy de la categoría 2».

11. La precarga: la inauguración del bar

La **precarga** es lo que la app trae de fábrica la primera vez que se abre: **Otros, 60 mesas, 14 alérgenos, Bebidas y el Agua a 1,50 €**. Como el móvil nuevo que ya trae la calculadora sin que nadie la instale.

- Salta **una sola vez**: cuando se crea el archivo de la base de datos. Solo hay una inauguración.
- Para repetirla hay que **desinstalar** la app (que borra también su base de datos) y volver a instalarla. No basta con darle a Run ►.
- Se hace con una **corrutina**: trabajo **por detrás**, como un pinche que monta las 60 mesas mientras el camarero sigue atendiendo en la barra.
- **Sorpresa del día**: Room **no abre la base de datos hasta que alguien le pide algo**, igual que la persiana del bar solo se sube cuando llega el primer cliente. Como hoy ninguna pantalla pide nada, la precarga no habría saltado nunca. Elegiste (P118) que `EntradaAko` suba la persiana **al arrancar**: el local está montado antes de que entre nadie.

12. El resultado

Desinstalamos, pulsaste Run ►, abriste el **Database Inspector** (View → Tool Windows → App Inspection) y ahí estaban: **Otros y Bebidas, el Agua, 60 mesas y los 14 alérgenos en el orden de la ley**. Había una tabla más, `room_master_table`, que la crea Room para apuntarse la versión. No es tuya.

13. Cosas de Android Studio que aprendiste

- **Colores del panel de archivos** (son de Git, no errores): verde = nuevo, azul = cambiado, rojo = Git no lo conoce. El rojo que importa es el que sale **dentro del código**.
- **Subrayado verde ondulado** = el corrector ortográfico en inglés. Ignóralo.
- **Nombres en gris y cursiva** = algo que todavía nadie usa.
- **Alt + Fn + I** (tu Insert es Fn + I) = crear archivo. Elige **Kotlin Class/File**, nunca «Java Class».
- **Shift + F6** = renombrar **lo que tengas seleccionado**. Haz clic primero en el archivo.
- **Ctrl + Alt + Y** = recargar desde el disco.
- **Alt + Enter** = arreglar los `import` que faltan.
- **Crear PR**: no se pulsa. Trabajamos solo en `main`.

Lo que viene: la S3

Las **consultas** de los DAOs («dame los platos visibles», «¿está ocupada la mesa 4?») y la **libreta de reglas** (`dominio`): el carrito, la calculadora, las comprobaciones y la picadora del PIN, con sus primeras pruebas automáticas.